```
# Exploratory data analysis
```

```
IRIS_HEADER = ["sepal_length", "sepal_width", "petal_length", "petal_width", "species"]

RAW_IRIS_DATA_LINES = """5.1,3.5,1.4,0.2,setosa
4.9,3.0,1.4,0.2,setosa
4.7,3.2,1.3,0.2,setosa
4.6,3.1,1.5,0.2,setosa
5.0,3.6,1.4,0.2,setosa
5.4,3.9,1.7,0.4,setosa
4.6,3.4,1.4,0.3,setosa
5.0,3.4,1.5,0.2,setosa
4.4,2.9,1.4,0.2,setosa
4.9,3.1,1.5,0.1,setosa
5.4,3.7,1.5,0.2,setosa
4.8,3.4,1.6,0.2,setosa
4.8,3.0,1.4,0.1,setosa
4.3,3.0,1.1,0.1,setosa
5.8,4.0,1.2,0.2,setosa
5.7,4.4,1.5,0.4,setosa
5.4,3.9,1.3,0.4,setosa
5.1,3.5,1.4,0.3,setosa
5.7,3.8,1.7,0.3,setosa
5.1,3.8,1.5,0.3,setosa
5.4,3.4,1.7,0.2,setosa
5.1,3.7,1.5,0.4,setosa
4.6,3.6,1.0,0.2,setosa
5.1,3.3,1.7,0.5,setosa
4.8,3.4,1.9,0.2,setosa
5.0,3.0,1.6,0.2,setosa
5.0,3.4,1.6,0.4,setosa
5.2,3.5,1.5,0.2,setosa
5.2,3.4,1.4,0.2,setosa
4.7,3.2,1.6,0.2,setosa
4.8,3.1,1.6,0.2,setosa
5.4,3.4,1.5,0.4,setosa
5.2,4.1,1.5,0.1,setosa
5.5,4.2,1.4,0.2,setosa
4.9,3.1,1.5,0.1,setosa
5.0,3.2,1.2,0.2,setosa
5.5,3.5,1.3,0.2,setosa
4.9,3.1,1.5,0.1,setosa
4.4,3.0,1.3,0.2,setosa
5.1,3.4,1.5,0.2,setosa
5.0,3.5,1.3,0.3,setosa
4.5,2.3,1.3,0.3,setosa
4.4,3.2,1.3,0.2,setosa
5.0,3.5,1.6,0.6,setosa
5.1,3.8,1.9,0.4,setosa
4.8,3.0,1.4,0.3,setosa
5.1,3.8,1.6,0.2,setosa
4.6,3.2,1.4,0.2,setosa
5.3,3.7,1.5,0.2,setosa
5.0,3.3,1.4,0.2,setosa
7.0,3.2,4.7,1.4,versicolor
6.4,3.2,4.5,1.5,versicolor
6.9,3.1,4.9,1.5,versicolor
5.5,2.3,4.0,1.3,versicolor
6.5,2.8,4.6,1.5,versicolor
5.7,2.8,4.5,1.3,versicolor
6.3,3.3,4.7,1.6,versicolor
4.9,2.4,3.3,1.0,versicolor
6.6,2.9,4.6,1.3,versicolor
5.2,2.7,3.9,1.4,versicolor
5.0,2.0,3.5,1.0,versicolor
5.9,3.0,4.2,1.5,versicolor
6.0,2.2,4.0,1.0,versicolor
6.1,2.9,4.7,1.4,versicolor
5.6,2.9,3.6,1.3,versicolor
6.7,3.1,4.4,1.4,versicolor
5.6,3.0,4.5,1.5,versicolor
5.8,2.7,4.1,1.0,versicolor
6.2,2.2,4.5,1.5,versicolor
5.6,2.5,3.9,1.1,versicolor
5.9,3.2,4.8,1.8,versicolor
6.1,2.8,4.0,1.3,versicolor
6.3,2.5,4.9,1.5,versicolor
6.1,2.8,4.7,1.2,versicolor
6.4,2.9,4.3,1.3,versicolor
6.6,3.0,4.4,1.4,versicolor
6.8,2.8,4.8,1.4,versicolor
6.7,3.0,5.0,1.7,versicolor
6.0,2.9,4.5,1.5,versicolor
5.7,2.6,3.5,1.0,versicolor
5.5,2.4,3.8,1.1,versicolor
5.5,2.4,3.7,1.0,versicolor
5.8,2.7,3.9,1.2,versicolor
6.0,2.7,5.1,1.6,versicolor
5.4,3.0,4.5,1.5,versicolor
6.0,3.4,4.5,1.6,versicolor
6.7,3.1,4.7,1.5,versicolor
6.3,2.3,4.4,1.3,versicolor
5.6,3.0,4.1,1.3,versicolor
5.5,2.5,4.0,1.3,versicolor
```

```
5.5,2.6,4.4,1.2,versicolor
6.1,3.0,4.6,1.4,versicolor
5.8,2.6,4.0,1.2,versicolor
5.0,2.3,3.3,1.0,versicolor
5.6,2.7,4.2,1.3,versicolor
5.7,3.0,4.2,1.2,versicolor
5.7,2.9,4.2,1.3,versicolor
6.2,2.9,4.3,1.3,versicolor
5.1,2.5,3.0,1.1,versicolor
5.7,2.8,4.1,1.3,versicolor
6.3,3.3,6.0,2.5,virginica
5.8,2.7,5.1,1.9,virginica
7.1,3.0,5.9,2.1,virginica
6.3,2.9,5.6,1.8,virginica
6.5,3.0,5.8,2.2,virginica
7.6,3.0,6.6,2.1,virginica
4.9,2.5,4.5,1.7,virginica
7.3,2.9,6.3,1.8,virginica
6.7,2.5,5.8,1.8,virginica
7.2,3.6,6.1,2.5,virginica
6.5,3.2,5.1,2.0,virginica
6.4,2.7,5.3,1.9,virginica
6.8,3.0,5.5,2.1,virginica
5.7,2.5,5.0,2.0,virginica
5.8,2.8,5.1,2.4,virginica
6.4,3.2,5.3,2.3,virginica
6.5,3.0,5.5,1.8,virginica
7.7,3.8,6.7,2.2,virginica
7.7,2.6,6.9,2.3,virginica
6.0,2.2,5.0,1.5,virginica
6.9,3.2,5.7,2.3,virginica
5.6,2.8,4.9,2.0,virginica
7.7,2.8,6.7,2.0,virginica
6.3,2.7,4.9,1.8,virginica
6.7,3.3,5.7,2.1,virginica
7.2,3.2,6.0,1.8,virginica
6.2,2.8,4.8,1.8,virginica
6.1,3.0,4.9,1.8,virginica
6.4,2.8,5.6,2.1,virginica
7.2,3.0,5.8,1.6,virginica
7.4,2.8,6.1,1.9,virginica
7.9,3.8,6.4,2.0,virginica
6.4,2.8,5.6,2.2,virginica
6.3,2.8,5.1,1.5,virginica
6.1,2.6,5.6,1.4,virginica
7.7,3.0,6.1,2.3,virginica
6.3,3.4,5.6,2.4,virginica
6.4,3.1,5.5,1.8,virginica
6.0,3.0,4.8,1.8,virginica
6.9,3.1,5.4,2.1,virginica
6.0,3.0,4.8,1.8,virginica
6.9,3.1,5.4,2.1,virginica
6.7,3.1,5.6,2.4,virginica
6.9,3.1,5.1,2.3,virginica
5.8,2.7,5.1,1.9,virginica
6.8,3.2,5.9,2.3,virginica
6.7,3.3,5.7,2.5,virginica
6.7,3.0,5.2,2.3,virginica
6.3,2.5,5.0,1.9,virginica
6.5,3.0,5.2,2.0,virginica
6.2,3.4,5.4,2.3,virginica
5.9,3.0,5.1,1.8,virginica"""

data_lines = [line.strip() for line in RAW_IRIS_DATA_LINES.splitlines() if line.strip()]

data_list = []

print("\n--- Exploring data structure ---")

header_line_str = ",".join(IRIS_HEADER)
print(f"\nFile header: {header_line_str}")

num_columns = len(IRIS_HEADER)
print(f"\nNumber of columns: {num_columns}")

column_names_str = ', '.join(IRIS_HEADER)
print(f"\nColumn names: {column_names_str}")

for line in data_lines:
    values = line.split(',')

    row_dict = {}
    for i in range(len(IRIS_HEADER)):
        col_name = IRIS_HEADER[i]
        value = values[i]

        if i < len(IRIS_HEADER) - 1:
            row_dict[col_name] = float(value)
        else:
            row_dict[col_name] = value

    data_list.append(row_dict)

num_rows = len(data_list)
print(f"\nNumber of rows of data (excluding header): {num_rows}")

print(f"\nData type of every column:")
if data_list:
    first_row = data_list[0]
```

```
    for col_name in IRIS_HEADER:
        data_type = type(first_row[col_name])
        print(f"  - Column '{col_name}': {data_type.__name__}")
else:
    print("  (Data list is empty, cannot determine data types.)")

print("\n--- Structure study complete ---")

def head(in_list, n):IRIS_HEADER = ["sepal_length", "sepal_width", "petal_length", "petal_width", "species"]

RAW_IRIS_DATA_LINES = """5.1,3.5,1.4,0.2,setosa
4.9,3.0,1.4,0.2,setosa
4.7,3.2,1.3,0.2,setosa
4.6,3.1,1.5,0.2,setosa
5.0,3.6,1.4,0.2,setosa
5.4,3.9,1.7,0.4,setosa
4.6,3.4,1.4,0.3,setosa
5.0,3.4,1.5,0.2,setosa
4.4,2.9,1.4,0.2,setosa
4.9,3.1,1.5,0.1,setosa
5.4,3.7,1.5,0.2,setosa
4.8,3.4,1.6,0.2,setosa
4.8,3.0,1.4,0.1,setosa
4.3,3.0,1.1,0.1,setosa
5.8,4.0,1.2,0.2,setosa
5.7,4.4,1.5,0.4,setosa
5.4,3.9,1.3,0.4,setosa
5.1,3.5,1.4,0.3,setosa
5.7,3.8,1.7,0.3,setosa
5.1,3.8,1.5,0.3,setosa
5.4,3.4,1.7,0.2,setosa
5.1,3.7,1.5,0.4,setosa
4.6,3.6,1.0,0.2,setosa
5.1,3.3,1.7,0.5,setosa
4.8,3.4,1.9,0.2,setosa
5.0,3.0,1.6,0.2,setosa
5.0,3.4,1.6,0.4,setosa
5.2,3.5,1.5,0.2,setosa
5.2,3.4,1.4,0.2,setosa
4.7,3.2,1.6,0.2,setosa
4.8,3.1,1.6,0.2,setosa
5.4,3.4,1.5,0.4,setosa
5.2,4.1,1.5,0.1,setosa
5.5,4.2,1.4,0.2,setosa
4.9,3.1,1.5,0.1,setosa
5.0,3.2,1.2,0.2,setosa
5.5,3.5,1.3,0.2,setosa
4.9,3.1,1.5,0.1,setosa
4.4,3.0,1.3,0.2,setosa
4.9,3.1,1.5,0.1,setosa
4.4,3.0,1.3,0.2,setosa
5.1,3.4,1.5,0.2,setosa
5.0,3.5,1.3,0.3,setosa
4.5,2.3,1.3,0.3,setosa
4.4,3.2,1.3,0.2,setosa
5.0,3.5,1.6,0.6,setosa
5.1,3.8,1.9,0.4,setosa
4.8,3.0,1.4,0.3,setosa
5.1,3.8,1.6,0.2,setosa
4.6,3.2,1.4,0.2,setosa
5.3,3.7,1.5,0.2,setosa
5.0,3.3,1.4,0.2,setosa
7.0,3.2,4.7,1.4,versicolor
6.4,3.2,4.5,1.5,versicolor
6.9,3.1,4.9,1.5,versicolor
5.5,2.3,4.0,1.3,versicolor
6.5,2.8,4.6,1.5,versicolor
5.7,2.8,4.5,1.3,versicolor
6.3,3.3,4.7,1.6,versicolor
4.9,2.4,3.3,1.0,versicolor
6.6,2.9,4.6,1.3,versicolor
5.2,2.7,3.9,1.4,versicolor
5.0,2.0,3.5,1.0,versicolor
5.9,3.0,4.2,1.5,versicolor
6.0,2.2,4.0,1.0,versicolor
6.1,2.9,4.7,1.4,versicolor
5.6,2.9,3.6,1.3,versicolor
6.7,3.1,4.4,1.4,versicolor
5.6,3.0,4.5,1.5,versicolor
5.8,2.7,4.1,1.0,versicolor
6.2,2.2,4.5,1.5,versicolor
5.6,2.5,3.9,1.1,versicolor
5.9,3.2,4.8,1.8,versicolor
6.1,2.8,4.0,1.3,versicolor
6.3,2.5,4.9,1.5,versicolor
6.1,2.8,4.7,1.2,versicolor
6.4,2.9,4.3,1.3,versicolor
6.6,3.0,4.4,1.4,versicolor
6.8,2.8,4.8,1.4,versicolor
6.7,3.0,5.0,1.7,versicolor
6.0,2.9,4.5,1.5,versicolor
5.7,2.6,3.5,1.0,versicolor
5.5,2.4,3.8,1.1,versicolor
5.5,2.4,3.7,1.0,versicolor
5.8,2.7,3.9,1.2,versicolor
6.0,2.7,5.1,1.6,versicolor
5.4,3.0,4.5,1.5,versicolor
6.0,3.4,4.5,1.6,versicolor
```

```
6.7,3.1,4.7,1.5,versicolor
6.3,2.3,4.4,1.3,versicolor
5.6,3.0,4.1,1.3,versicolor
5.5,2.5,4.0,1.3,versicolor
5.5,2.6,4.4,1.2,versicolor
6.1,3.0,4.6,1.4,versicolor
5.8,2.6,4.0,1.2,versicolor
5.0,2.3,3.3,1.0,versicolor
5.6,2.7,4.2,1.3,versicolor
5.7,3.0,4.2,1.2,versicolor
5.7,2.9,4.2,1.3,versicolor
6.2,2.9,4.3,1.3,versicolor
5.1,2.5,3.0,1.1,versicolor
5.7,2.8,4.1,1.3,versicolor
6.3,3.3,6.0,2.5,virginica
5.8,2.7,5.1,1.9,virginica
7.1,3.0,5.9,2.1,virginica
6.3,2.9,5.6,1.8,virginica
6.5,3.0,5.8,2.2,virginica
7.6,3.0,6.6,2.1,virginica
4.9,2.5,4.5,1.7,virginica
7.3,2.9,6.3,1.8,virginica
6.7,2.5,5.8,1.8,virginica
7.2,3.6,6.1,2.5,virginica
6.5,3.2,5.1,2.0,virginica
6.4,2.7,5.3,1.9,virginica
6.8,3.0,5.5,2.1,virginica
5.7,2.5,5.0,2.0,virginica
5.8,2.8,5.1,2.4,virginica
6.4,3.2,5.3,2.3,virginica
6.5,3.0,5.5,1.8,virginica
7.7,3.8,6.7,2.2,virginica
7.7,2.6,6.9,2.3,virginica
6.0,2.2,5.0,1.5,virginica
6.9,3.2,5.7,2.3,virginica
5.6,2.8,4.9,2.0,virginica
7.7,2.8,6.7,2.0,virginica
6.3,2.7,4.9,1.8,virginica
6.7,3.3,5.7,2.1,virginica
7.2,3.2,6.0,1.8,virginica
6.2,2.8,4.8,1.8,virginica
6.1,3.0,4.9,1.8,virginica
6.4,2.8,5.6,2.1,virginica
7.2,3.0,5.8,1.6,virginica
7.4,2.8,6.1,1.9,virginica
7.9,3.8,6.4,2.0,virginica
6.4,2.8,5.6,2.2,virginica
6.3,2.8,5.1,1.5,virginica
6.1,2.6,5.6,1.4,virginica
7.7,3.0,6.1,2.3,virginica
6.3,3.4,5.6,2.4,virginica
6.4,3.1,5.5,1.8,virginica
6.0,3.0,4.8,1.8,virginica
6.9,3.1,5.4,2.1,virginica
6.7,3.1,5.6,2.4,virginica
6.9,3.1,5.1,2.3,virginica
5.8,2.7,5.1,1.9,virginica
6.8,3.2,5.9,2.3,virginica
6.7,3.3,5.7,2.5,virginica
6.7,3.0,5.2,2.3,virginica
6.3,2.5,5.0,1.9,virginica
6.5,3.0,5.2,2.0,virginica
6.2,3.4,5.4,2.3,virginica
5.9,3.0,5.1,1.8,virginica"""

WIDTHS = [15, 12, 13, 12, 10]
SEP = " "

print("\n--- data parsing and structure exploration ---")

data_lines = [line.strip() for line in RAW_IRIS_DATA_LINES.splitlines() if line.strip()]
data_list = []

header_line_str = ",".join(IRIS_HEADER)
print(f"\nfile header: {header_line_str}")

num_columns = len(IRIS_HEADER)
print(f"\nnumber of columns: {num_columns}")

print(f"\ncolumn names: {header_line_str}")

for line in data_lines:
    values = line.split(',')

    row_dict = {}
    for i in range(len(IRIS_HEADER)):
        col_name = IRIS_HEADER[i]
        value = values[i]

        if i < len(IRIS_HEADER) - 1:
            row_dict[col_name] = float(value)
        else:
            row_dict[col_name] = value

    data_list.append(row_dict)

num_rows = len(data_list)
```

```
    print(f"\nnumber of rows of data (excluding header): {num_rows}")

    print(f"\ndata type of every column:")
    if data_list:
        first_row = data_list[0]
        for col_name in IRIS_HEADER:
            data_type = type(first_row[col_name])
            print(f"  - column '{col_name}': {data_type.__name__}")

    def head(data_list, n=5):
        """
        (a) Prints the header plus the first 'n' rows of the dataset.
        """
        print(f"\n--- head (first {n} rows) ---")

        print(f"{IRIS_HEADER[0]:<{WIDTHS[0]}}{SEP}"
              f"{IRIS_HEADER[1]:<{WIDTHS[1]}}{SEP}"
              f"{IRIS_HEADER[2]:<{WIDTHS[2]}}{SEP}"
              f"{IRIS_HEADER[3]:<{WIDTHS[3]}}{SEP}"
              f"{IRIS_HEADER[4]:<{WIDTHS[4]}}")

        print("-" * sum(WIDTHS) + "-" * (len(WIDTHS) - 1))

        rows_to_print = min(n, len(data_list))

        for i in range(rows_to_print):
            row = data_list[i]

            print(f"{row['sepal_length']:<{WIDTHS[0]}.2f}{SEP}"
                  f"{row['sepal_width']:<{WIDTHS[1]}.2f}{SEP}"
                  f"{row['petal_length']:<{WIDTHS[2]}.2f}{SEP}"
                  f"{row['petal_width']:<{WIDTHS[3]}.2f}{SEP}"
                  f"{row['species']:<{WIDTHS[4]}}")


    def tail(data_list, n=5):
        """
        Prints the header plus the last 'n' rows of the dataset.
        """
        print(f"\n--- tail (last {n} rows) ---")

        print(f"{IRIS_HEADER[0]:<{WIDTHS[0]}}{SEP}"
              f"{IRIS_HEADER[1]:<{WIDTHS[1]}}{SEP}"
              f"{IRIS_HEADER[2]:<{WIDTHS[2]}}{SEP}"
              f"{IRIS_HEADER[3]:<{WIDTHS[3]}}{SEP}"
              f"{IRIS_HEADER[4]:<{WIDTHS[4]}}")

        print("-" * sum(WIDTHS) + "-" * (len(WIDTHS) - 1))

        data_size = len(data_list)
        start_index = max(0, data_size - n)

        for i in range(start_index, data_size):
            row = data_list[i]

            print(f"{row['sepal_length']:<{WIDTHS[0]}.2f}{SEP}"
                  f"{row['sepal_width']:<{WIDTHS[1]}.2f}{SEP}"
                  f"{row['petal_length']:<{WIDTHS[2]}.2f}{SEP}"
                  f"{row['petal_width']:<{WIDTHS[3]}.2f}{SEP}"
                  f"{row['species']:<{WIDTHS[4]}}")

    head(data_list)
    tail(data_list)

    def head(in_list, n):
        print(f"\n--- Head of data (First {n} rows) ---")

        if not in_list:
            print("Error: The input list is empty.")
            return

        if n > len(in_list):
            print(f"Error: Requested {n} items, but the list only contains {len(in_list)} items.")
            return

        header = list(in_list[0].keys())

        header_str = " | ".join(f"{col:<15}" for col in header)
        print(header_str)
        print("-" * (len(header_str) + len(header) * 2))

        for i in range(n):
            row = in_list[i]
            row_values = [str(v) for v in row.values()]
            row_str = " | ".join(f"{val:<15}" for val in row_values)
            print(row_str)


    def tail(in_list, n):
        print(f"\n--- Tail of data (Last {n} rows) ---")

        if not in_list:
            print("Error: The input list is empty.")
            return

        if n > len(in_list):
            print(f"Error: Requested {n} items, but the list only contains {len(in_list)} items.")
```

```
            return

        header = list(in_list[0].keys())

        header_str = " | ".join(f"{col:<15}" for col in header)
        print(header_str)
        print("-" * (len(header_str) + len(header) * 2))

        start_index = len(in_list) - n

        for i in range(start_index, len(in_list)):
            row = in_list[i]
            row_values = [str(v) for v in row.values()]
            row_str = " | ".join(f"{val:<15}" for val in row_values)
            print(row_str)

    head(data_list, 5)

    from scratch.statistics import mean, median, mode, quantile, interquartile_range, data_range
    from collections import Counter

    numerical_features = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
    categorical_features = ['species']

    print("\n--- descriptive statistics for numerical features ---")

    for feature in numerical_features:
        values = [row[feature] for row in data_list]

        print(f"\nFeature: {feature}")

        try:
            feature_mean = mean(values)
            print(f"  - Mean: {feature_mean:.2f}")
        except Exception as e:
            print(f"  - Mean: could not calculate ({e})")

        try:
            feature_median = median(values)
            print(f"  - Median: {feature_median:.2f}")
        except Exception as e:
            print(f"  - Median: could not calculate ({e})")

        try:
            feature_mode = mode(values)
            print(f"  - Mode: {feature_mode}")
        except Exception as e:
            print(f"  - Mode: could not calculate ({e})")
     try:
            feature_range = data_range(values)
            print(f"  - Range: {feature_range:.2f}")
        except Exception as e:
            print(f"  - Range: could not calculate ({e})")

        try:
            feature_iqr = interquartile_range(values)
            print(f"  - IQR: {feature_iqr:.2f}")
        except Exception as e:
            print(f"  - IQR: could not calculate ({e})")

    print("\n--- end of numerical feature statistics ---")

    print("\n--- descriptive statistics for categorical features ---")

    for feature in categorical_features:
        values = [row[feature] for row in data_list]
        value_counts = Counter(values)

        print(f"\nFeature: {feature}")
        print(f"  - number of unique values: {len(value_counts)}")
        for value, count in value_counts.items():
            print(f"    - '{value}': {count} occurrences")

    print("\n--- end of categorical feature statistics ---")
```

```
    from scratch.statistics import correlation
    import matplotlib.pyplot as plt

    sepal_length_values = [row['sepal_length'] for row in data_list]
    sepal_width_values = [row['sepal_width'] for row in data_list]

    try:
        corr_coefficient = correlation(sepal_length_values, sepal_width_values)
        print(f"\n--- Correlation between sepal_length and sepal_width ---")
        print(f"The correlation coefficient between sepal_length and sepal_width is: {corr_coefficient:.4f}")

        if abs(corr_coefficient) < 0.3:
            conclusion = "There is a weak (or no) linear relationship between sepal_length and sepal_width."
        elif abs(corr_coefficient) < 0.7:
            conclusion = "There is a moderate linear relationship between sepal_length and sepal_width."
        else:
            conclusion = "There is a strong linear relationship between sepal_length and sepal_width."

        print(f"Conclusion: {conclusion}")
```

```
except Exception as e:
    print(f"Error calculating correlation: {e}")

print("\n--- Scatter plot of sepal_length vs sepal_width ---")
plt.figure(figsize=(8, 6))
plt.scatter(sepal_length_values, sepal_width_values, alpha=0.7)
plt.title('Scatter plot of sepal length vs sepal width')
plt.xlabel('Sepal length (cm)')
plt.ylabel('Sepal width (cm)')
plt.grid(True)
plt.show()
```

```
from scratch.statistics import correlation
import matplotlib.pyplot as plt

sepal_length_values = [row['sepal_length'] for row in data_list]
petal_length_values = [row['petal_length'] for row in data_list]

try:
    corr_coefficient = correlation(sepal_length_values, petal_length_values)
    print(f"\n--- Correlation between sepal_length and petal_length ---")
    print(f"The correlation coefficient between sepal_length and petal_length is: {corr_coefficient:.4f}")

    if abs(corr_coefficient) < 0.3:
        conclusion = "There is a weak (or no) linear relationship between sepal_length and petal_length."
    elif abs(corr_coefficient) < 0.7:
        conclusion = "There is a moderate linear relationship between sepal_length and petal_length."
    else:
        conclusion = "There is a strong linear relationship between sepal_length and petal_length."

    print(f"Conclusion: {conclusion}")

except Exception as e:
    print(f"Error calculating correlation: {e}")

print("\n--- Scatter plot of sepal_length vs petal_length ---")
plt.figure(figsize=(8, 6))
plt.scatter(sepal_length_values, petal_length_values, alpha=0.7)
plt.title('Scatter plot of sepal length vs petal length')
plt.xlabel('Sepal length (cm)')
plt.ylabel('Petal length (cm)')
plt.grid(True)
plt.show()
```

```
from collections import Counter
import matplotlib.pyplot as plt

species_values = [row['species'] for row in data_list]

species_counts = Counter(species_values)

species_names = list(species_counts.keys())
species_occurrences = list(species_counts.values())

plt.figure(figsize=(8, 6))
plt.bar(species_names, species_occurrences, color=['lightgreen'])
plt.title('Distribution of iris species')
plt.xlabel('Species')
plt.ylabel('Number of occurrences')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```